

Multi-Layer Perceptron and Logistic Regression

1 Perceptron and Multi-Layer Perceptron for the XOR Problem

1.1 Single-Layer Perceptron

The *Perceptron*, introduced by Rosenblatt (1958), is the simplest artificial neuron. It takes an input vector $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$, applies a set of weights $\mathbf{w} = [w_1, w_2, \dots, w_n]^T$, adds a bias b , and passes the result through an activation function:

$$y = f(\mathbf{w}^T \mathbf{x} + b) \quad (1)$$

where $f(\cdot)$ is typically a **step function**:

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

Learning Algorithm

The perceptron learning rule updates weights whenever a misclassification occurs.

- (i) Initialize $\mathbf{w} = \mathbf{0}$ and $b = 0$.
- (ii) For each training sample $(\mathbf{x}_i, y_i)$:

$$\hat{y}_i = f(\mathbf{w}^T \mathbf{x}_i + b)$$

$$\text{If } y_i \neq \hat{y}_i : \quad \mathbf{w} \leftarrow \mathbf{w} + \eta(y_i - \hat{y}_i)\mathbf{x}_i, \quad b \leftarrow b + \eta(y_i - \hat{y}_i)$$

- (iii) Repeat until convergence.

This algorithm converges if the dataset is *linearly separable*. The resulting decision boundary $\mathbf{w}^T \mathbf{x} + b = 0$ divides the input space into two classes.

1.2 Limitation: XOR Problem

The XOR (exclusive-or) function is defined as:

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

These points cannot be separated by a single linear boundary. Therefore, a single-layer perceptron **fails** on the XOR function. This motivated the introduction of multi-layer architectures.

1.3 Multi-Layer Perceptron (MLP)

An **MLP** introduces one or more hidden layers, enabling the network to model nonlinear relationships. Each neuron performs a nonlinear transformation:

$$a_j = f\left(\sum_i w_{ji}x_i + b_j\right) \quad (2)$$

Common activation functions include sigmoid, tanh, and ReLU.

Two-Layer Network for XOR

We can solve XOR with:

- 2 input neurons (x_1, x_2),
- 2 hidden neurons (h_1, h_2),
- 1 output neuron.

$$h_1 = f(w_{11}x_1 + w_{12}x_2 + b_1), \quad (3)$$

$$h_2 = f(w_{21}x_1 + w_{22}x_2 + b_2), \quad (4)$$

$$y = f(v_1h_1 + v_2h_2 + b_3) \quad (5)$$

Example of Weights

With $f(z) = 1$ if $z \geq 0$ else 0, one valid configuration is:

Neuron	Weights	Bias	Description
h_1	(1, 1)	-1.5	Detects ($x_1 + x_2 \geq 1.5$)
h_2	(1, 1)	-0.5	Detects ($x_1 + x_2 \geq 0.5$)
Output	(1, -1)	-0.5	Combines hidden signals

Verification:

x_1	x_2	h_1	h_2	y	XOR Truth
0	0	0	0	0	0
0	1	0	1	1	1
1	0	0	1	1	1
1	1	1	1	0	0

Thus, the two-layer perceptron successfully models XOR.

Architecture Description

The network architecture consists of:

- **Input layer:** 2 neurons (x_1, x_2)
- **Hidden layer:** 2 neurons (h_1, h_2) with step activation
- **Output layer:** 1 neuron (y) with step activation
- **Connections:** Fully connected between adjacent layers

1.4 Learning via Backpropagation

For general nonlinear problems, weights are learned by minimizing a loss function such as the mean squared error:

$$L = \frac{1}{2} \sum_i (y_i - \hat{y}_i)^2 \quad (6)$$

Gradients are propagated backward through the network using the **chain rule**, allowing updates:

$$w_{ij}^{(l)} \leftarrow w_{ij}^{(l)} - \eta \frac{\partial L}{\partial w_{ij}^{(l)}}$$

This process—called *backpropagation*—enables efficient optimization via stochastic gradient descent.

1.5 Summary

Property	Single-Layer Perceptron	Multi-Layer Perceptron
Activation	Step (linear boundary)	Nonlinear (sigmoid, ReLU)
Decision boundary	Linear	Nonlinear
Learning rule	Perceptron update	Backpropagation
Can solve XOR?	No	Yes
Minimum hidden neurons for XOR	N/A	2

1.6 References

1. F. Rosenblatt, “*The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain*”, Psychological Review, 1958.

2. D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “*Learning Representations by Back-Propagating Errors*”, Nature, 1986.
3. S. Haykin, *Neural Networks and Learning Machines*, 3rd Edition, Pearson, 2008.
4. M. Minsky and S. Papert, *Perceptrons: An Introduction to Computational Geometry*, MIT Press, 1969.

2 Logistic Regression

2.1 Overview

Logistic Regression is a fundamental algorithm for binary classification that models the *probability* that a given input belongs to a particular class. While the perceptron performs hard classification via a threshold function, logistic regression uses a **sigmoid (logistic) function** to output smooth probabilities between 0 and 1.

The model is linear in its parameters but nonlinear in its output, making it interpretable and widely applicable.

2.2 Model Formulation

Let $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$ denote an input feature vector and $y \in \{0, 1\}$ the target label. We define a linear predictor:

$$z = \mathbf{w}^T \mathbf{x} + b \tag{7}$$

where $\mathbf{w} = [w_1, w_2, \dots, w_n]^T$ are the model parameters (weights) and b is the bias.

Instead of using a step function, we apply the *sigmoid* transformation to map z to a probability:

$$p(y = 1 \mid \mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}. \tag{8}$$

Hence,

$$p(y = 0 \mid \mathbf{x}) = 1 - \sigma(z).$$

2.3 Decision Boundary

The classifier assigns the label $\hat{y} = 1$ if $p(y = 1 \mid \mathbf{x}) \geq 0.5$, equivalently when:

$$\mathbf{w}^T \mathbf{x} + b \geq 0.$$

Therefore, logistic regression is a *linear classifier* in feature space, but its probabilistic interpretation makes it more robust and informative than a perceptron.

2.4 Likelihood and Loss Function

Given N independent samples $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$, we model the probability of observing the labels as:

$$P(y_1, \dots, y_N \mid \mathbf{X}, \mathbf{w}, b) = \prod_{i=1}^N \sigma(z_i)^{y_i} [1 - \sigma(z_i)]^{1-y_i}, \quad (9)$$

where $z_i = \mathbf{w}^T \mathbf{x}_i + b$.

Maximizing this likelihood is equivalent to minimizing the negative log-likelihood (cross-entropy loss):

$$\mathcal{L}(\mathbf{w}, b) = - \sum_{i=1}^N \left[y_i \log(\sigma(z_i)) + (1 - y_i) \log(1 - \sigma(z_i)) \right]. \quad (10)$$

2.5 Gradient Descent Updates

To minimize $\mathcal{L}$, we compute partial derivatives with respect to the parameters.

Since $\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))$, we have:

$$\frac{\partial \mathcal{L}}{\partial w_j} = \sum_{i=1}^N (\sigma(z_i) - y_i) x_{ij}, \quad (11)$$

$$\frac{\partial \mathcal{L}}{\partial b} = \sum_{i=1}^N (\sigma(z_i) - y_i). \quad (12)$$

The **gradient descent** update rule for iteration t is:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \sum_{i=1}^N (\sigma(z_i) - y_i) \mathbf{x}_i, \quad (13)$$

$$b^{(t+1)} = b^{(t)} - \eta \sum_{i=1}^N (\sigma(z_i) - y_i), \quad (14)$$

where $\eta > 0$ is the learning rate.

In practice, stochastic or mini-batch gradient descent is often used for scalability:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta (\sigma(z_i) - y_i) \mathbf{x}_i.$$

2.6 Regularization

To prevent overfitting and control model complexity, a regularization term can be added to the loss:

- **L2 Regularization (Ridge):**

$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \frac{\lambda}{2} \|\mathbf{w}\|_2^2$$

- **L1 Regularization (Lasso):**

$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \lambda \|\mathbf{w}\|_1$$

These penalties encourage weight shrinkage (L2) or sparsity (L1), improving generalization.

2.7 Geometric and Probabilistic Interpretation

- The decision boundary $\mathbf{w}^T \mathbf{x} + b = 0$ divides the feature space into two regions corresponding to $p(y = 1) > 0.5$ and $p(y = 1) < 0.5$.
- The slope of the sigmoid controls the model's *confidence*: points near the boundary yield probabilities close to 0.5, while points far from it yield near-certain predictions.
- Logistic regression can be derived as a special case of a **Generalized Linear Model (GLM)** with the *logit link function*:

$$\log \frac{p(y = 1 \mid \mathbf{x})}{1 - p(y = 1 \mid \mathbf{x})} = \mathbf{w}^T \mathbf{x} + b.$$

2.8 Advantages and Limitations

Aspect	Advantages	Limitations
Interpretability	Coefficients indicate direction and strength of feature influence.	Assumes linear decision boundary in feature space.
Training	Convex loss ensures a unique global minimum.	Sensitive to multicollinearity and outliers.
Probabilistic Output	Produces calibrated probabilities.	May underperform when features interact nonlinearly.

2.9 Relationship to the Perceptron

Both the perceptron and logistic regression compute a weighted sum $\mathbf{w}^T \mathbf{x} + b$. However:

- The perceptron uses a **discontinuous step function**, producing hard decisions.
- Logistic regression uses a **smooth sigmoid function**, producing probabilistic outputs.
- The perceptron minimizes misclassification error (non-convex), whereas logistic regression minimizes a convex cross-entropy loss, making it more stable and differentiable.

2.10 Summary

- Logistic regression is a probabilistic linear classifier.
- It uses the sigmoid function to map real-valued inputs to $(0, 1)$.
- Model parameters are estimated by minimizing cross-entropy loss via gradient descent.

- Regularization enhances generalization and prevents overfitting.

2.11 References

1. D. W. Hosmer, S. Lemeshow, and R. X. Sturdivant, *Applied Logistic Regression*, Wiley, 2013.
2. T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, Springer, 2009.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

3 Comparison Between Perceptron and Logistic Regression

3.1 Motivation

Both the **Perceptron** and **Logistic Regression** are linear classifiers that form the foundation of modern neural networks and statistical learning methods. While they share the same functional form $\mathbf{w}^T \mathbf{x} + b$, they differ fundamentally in their *learning objectives*, *interpretations*, and *optimization properties*.

This section presents a detailed comparison from multiple perspectives: geometric, probabilistic, and algorithmic.

3.2 Model Formulation

Both models compute a weighted linear combination:

$$z = \mathbf{w}^T \mathbf{x} + b,$$

but their output functions differ:

	Perceptron	Logistic Regression
Output function	$f(z) = \text{sign}(z)$	$f(z) = \sigma(z) = \frac{1}{1 + e^{-z}}$
Output range	$\{-1, +1\}$ or $\{0, 1\}$	$(0, 1)$
Interpretation	Hard class label	Class probability $p(y = 1 \mid \mathbf{x})$

3.3 Learning Objective

The perceptron minimizes the *misclassification error* via an update rule that adjusts weights when predictions are incorrect. It effectively tries to find any hyperplane that separates the data.

In contrast, logistic regression minimizes the *negative log-likelihood (cross-entropy loss)*:

$$\mathcal{L}(\mathbf{w}, b) = - \sum_i \left[y_i \log(\sigma(z_i)) + (1 - y_i) \log(1 - \sigma(z_i)) \right],$$

which is a **smooth and convex** objective.

3.4 Optimization and Convergence

- **Perceptron:** Uses a simple update rule $\mathbf{w} \leftarrow \mathbf{w} + \eta y_i \mathbf{x}_i$ for misclassified points. Convergence is guaranteed only if the data are linearly separable (Perceptron Convergence Theorem). When data are not separable, it oscillates indefinitely.
- **Logistic Regression:** Uses gradient descent or quasi-Newton optimization (e.g., BFGS) to minimize a convex loss. It always converges to a unique global minimum regardless of separability.

Thus, logistic regression provides a more stable and principled optimization framework.

3.5 Geometric Perspective

Both methods produce a linear decision boundary:

$$\mathbf{w}^T \mathbf{x} + b = 0.$$

However:

- In the perceptron, the boundary is determined by any separating hyperplane; all misclassified points drive weight updates equally.
- In logistic regression, each point influences the model according to its probability error $\sigma(z_i) - y_i$. Points near the boundary (where $\sigma(z_i) \approx 0.5$) contribute more strongly to learning.

Hence, logistic regression assigns *soft importance* to uncertain samples, leading to smoother decision boundaries.

3.6 Probabilistic Interpretation

- The perceptron is a deterministic rule: it classifies based on the sign of a linear function, with no probabilistic meaning.
- Logistic regression directly models the **posterior probability** of class membership:

$$p(y = 1 \mid \mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}.$$

It therefore allows threshold adjustment (e.g., 0.3, 0.7) and integration with probabilistic frameworks such as Bayesian inference and likelihood-based learning.

3.7 Loss Function Relationship

The perceptron loss and logistic loss share structural similarity but differ in smoothness:

	Perceptron Loss	Logistic Loss
Definition	$L_{\text{perc}}(y, z) = \max(0, -yz)$	$L_{\text{log}}(y, z) = \log(1 + e^{-yz})$
Shape	Piecewise linear	Smooth and convex
Gradient	Discontinuous at $yz = 0$	Continuous everywhere
Optimization	Non-convex	Convex

The logistic loss can be viewed as a *smooth approximation* to the perceptron loss, improving convergence and generalization.

3.8 Interpretability and Output Calibration

- Logistic regression coefficients quantify the change in the log-odds of the outcome per unit change in the feature, enabling interpretable models.
- Perceptron weights lack probabilistic meaning and mainly represent directional importance for separation.
- Logistic regression produces **calibrated probabilities**, allowing for uncertainty estimation; the perceptron produces only binary outputs.

3.9 Robustness and Regularization

- Logistic regression easily accommodates regularization terms such as ℓ_1 or ℓ_2 penalties.
- The perceptron algorithm lacks a natural probabilistic or penalized framework, although extensions like the *margin perceptron* introduce such controls.

3.10 When to Use Each

- **Perceptron:** Simple, interpretable, and efficient for linearly separable data. Useful for online or streaming contexts where fast updates are preferred.
- **Logistic Regression:** Preferred when probabilistic outputs, convex optimization, or regularization are required. It performs better in non-separable and noisy settings.

3.11 Summary Table

Aspect	Perceptron	Logistic Regression
Activation	Step function	Sigmoid function
Output	Class label ($\{-1, +1\}$)	Probability $(0, 1)$
Loss Function	Misclassification / hinge-like	Cross-entropy (log-likelihood)
Objective Type	Non-convex, discontinuous	Smooth and convex
Convergence	Only for separable data	Always converges (convex)
Interpretation	Geometric separator	Probabilistic model (GLM)
Regularization	Not standard	Easily incorporated (ℓ_1, ℓ_2)
Gradient	Discontinuous updates	Continuous, differentiable
Output Certainty	Hard decision	Confidence level via probability

3.12 Key Insight

The perceptron can be viewed as an *ancestor* of logistic regression: it learns a separating hyperplane via thresholding, while logistic regression smooths this threshold into a differentiable sigmoid, allowing gradient-based optimization and probabilistic interpretation. In essence, logistic regression is a *continuous and probabilistic generalization* of the perceptron.